Country Lookup - Spring Boot Assignment

A REST web service that accepts latitude and longitude and returns the country name and two-letter country code using OpenStreetMap's Nominatim reverse-geocoding service.

Submission status

Part	Implementation	Verification / remaining work
1 - REST API	Endpoint, validation, WebClient, error handling, caching and automated tests included	Build attempted but blocked before compilation: Maven dependencies could not be downloaded from this workspace. Tests are included but not executed.
2 - Docker	Multi-stage Dockerfile included	Docker is unavailable in the preparation workspace. Build/run not verified; final image size not measured.
3 - Kubernetes	kind instructions, two application replicas, Service and shared provider gateway included	Cluster and host-browser demonstration not executed. Mandatory screencast not recorded or uploaded.
Documentation	This Markdown README and matching PDF included	Update measured results and video link, then regenerate PDF before submission.

This is a prepared implementation, not a claim that all assignment parts have been demonstrated successfully. No Git or GitHub workflow is required.

Requirements

• Java JDK 17 or newer for local development (Java 17 is used by Docker).

• Maven Wrapper is included; a separate Maven installation is not required.

• Internet access for Maven dependencies, container images and live geocoding.

• Docker Desktop running with Linux containers for Parts 2 and 3.

• kind and kubectl installed and available in your terminal for Part 3.

• Python 3 and ReportLab only if regenerating the README PDF.

Official installation guides:

• Docker Desktop for Windows: https://docs.docker.com/desktop/setup/install/windows-install/

• kind: https://kind.sigs.k8s.io/docs/user/quick-start/

• kubectl for Windows: https://kubernetes.io/docs/tasks/tools/install-kubectl-windows/

Commands below use Windows PowerShell. Run them from the project folder containing `pom.xml`. For example, if the project is at the original location:

```
Set-Location "C:\Users\Digital Inclusion\Desktop\cloud\demo"
```

Part 1 - Build and run locally

Check Java, build the application and run the tests:

```
java -version
.\mvnw.cmd clean verify
```

Run the packaged application:

```
java -jar target/country-lookup.jar
```

On Linux/macOS, use `./mvnw clean verify` instead. If needed, first run `chmod +x mvnw`.

REST endpoint

```
GET /api/country?latitude=<number>&longitude=<number>
```

Example URL for Luxembourg City:

```
http://localhost:8080/api/country?latitude=49.6116&longitude=6.1319
```

Open that URL in a browser or use another terminal:

```
curl.exe -i "http://localhost:8080/api/country?latitude=49.6116&longitude=6.1319"
```

Expected successful response, subject to live provider availability:

```
{
  "country": "Luxembourg",
  "countryCode": "LU",
  "attribution": "Data © OpenStreetMap contributors, ODbL 1.0: https://www.openstreetmap.org/copyright"
}
```

The country code is returned in uppercase. Country names are requested in English. The attribution field credits the data source.

Input and error handling

Situation	HTTP response
Valid coordinates and usable provider response	200 OK
Missing, blank or nonnumeric latitude/longitude	400 Bad Request
Latitude outside [-90, 90] or longitude outside [-180, 180]	400 Bad Request
NaN or Infinity	400 Bad Request
Provider HTTP error, timeout, connection failure or invalid JSON	502 Bad Gateway
Empty response or missing/invalid country information	502 Bad Gateway

The coordinate limits are inclusive. Valid coordinates can still produce 502 when no country information is available, for example over the ocean. A simulated test accepting a coordinate boundary does not prove that the real provider has country data there.

Try invalid requests:

```
curl.exe -i "http://localhost:8080/api/country?latitude=91&longitude=6"
curl.exe -i "http://localhost:8080/api/country?latitude=49"
curl.exe -i "http://localhost:8080/api/country?latitude=abc&longitude=6"
```

Errors are JSON objects containing `status` and `error`. Invalid input is rejected before contacting the provider.

External provider configuration and policy

The public Nominatim service is for light, user-triggered use. Its policy requires an identifying User-Agent, attribution, caching of repeated queries and an application-wide maximum of one request per second. Do not run load tests against it.

Read the policy before live use: https://operations.osmfoundation.org/policies/nominatim/

Each application instance serialises uncached requests with at least 1.1 seconds between starts. Successful results are cached for 24 hours, up to 256 entries. Errors are not cached. The Kubernetes configuration adds one shared gateway so the two app replicas do not each independently send one request per second to the public provider. Keep this gateway at one replica and avoid simultaneously running other copies of this application against the public provider.

The default request identifier is `CountryLookupStudentAssignment/1.0`. You can customise it before starting the app:

```
$env:GEOCODING_USER_AGENT = "CountryLookupAssignment/1.0"
java -jar target/country-lookup.jar
```

Environment settings:

Variable	Default	Purpose
GEOCODING_BASE_URL	https://nominatim.openstreetmap.org	Alternative compatible provider or test server
GEOCODING_USER_AGENT	CountryLookupStudentAssignment/1.0	Identifies this application
GEOCODING_TIMEOUT_MS	15000	Provider request timeout in milliseconds

Nominatim finds suitable mapped objects near the coordinates; it is not an exact legal border-determination service. Results close to borders may reflect nearby mapped objects.

Reproducible 502 demonstration

Stop the running app with Ctrl+C. In PowerShell, point it to an unused local port:

```
$env:GEOCODING_BASE_URL = "http://127.0.0.1:1"
java -jar target/country-lookup.jar
```

In another terminal, call the valid example URL. Expected status: 502, because the provider connection fails. Stop the app and restore the default before further work:

```
Remove-Item Env:GEOCODING_BASE_URL -ErrorAction SilentlyContinue
```

Automated tests

`CountryApiTests` starts the real Spring Boot application on a random port and uses a local simulated HTTP provider. It checks success, provider query parameters, required inputs, inclusive ranges, invalid numbers, provider HTTP errors, malformed/unusable responses, timeout and caching. It does not send test traffic to OpenStreetMap.

```
.\mvnw.cmd test
```

Part 2 - Docker

Stop the locally running Java application before using host port 8080.

Build the image from the project root:

```
docker build -t country-lookup:1.0 .
```

The build stage uses Maven and JDK 17 and runs the tests. The runtime stage contains only the application JAR and an Alpine-based JRE. It runs as a non-root user. Maven, source files and build tools are not copied into the final image.

Run:

```
docker run --rm --name country-lookup -p 8080:8080 country-lookup:1.0
```

Open the same example URL:

```
http://localhost:8080/api/country?latitude=49.6116&longitude=6.1319
```

In a second terminal, measure the actual image size:

```
docker image ls country-lookup:1.0
docker image inspect country-lookup:1.0 --format '{{.Size}}'
```

Measured final image size: Not measured yet. Replace this statement with the actual `docker image ls` size and `docker image inspect` byte count, including your CPU architecture if relevant.

Stop the container before the cluster demonstration:

```
docker stop country-lookup
```

The goal is a small practical runtime image; no claim is made that this is the smallest possible Java image.

Part 3 - Local Kubernetes with kind

1. Check prerequisites and create the cluster

Docker Desktop must be running in Linux-container mode. The following commands assume Docker, kind and kubectl are already installed using the guides above.

```
docker version
kind version
kubectl version --client
kind create cluster --name country-assignment
kubectl --context kind-country-assignment get nodes
```

2. Build and load the custom image

Skip the build only if you already built exactly this image from the current source.

```
docker build -t country-lookup:1.0 .
kind load docker-image country-lookup:1.0 --name country-assignment
```

The manifest uses `imagePullPolicy: Never`, ensuring Kubernetes uses the locally loaded image rather than a registry.

3. Deploy the shared geocoding gateway

The gateway uses a single shared rate-limit bucket and response cache. It forwards requests to Nominatim over verified HTTPS. It is an internal dependency, not another public business endpoint.

```
kubectl --context kind-country-assignment create configmap geocoding-gateway-config --from-
file=nginx.conf=gateway/nginx.conf --dry-run=client -o yaml | kubectl --context kind-country-assignment
apply -f -
kubectl --context kind-country-assignment apply -f k8s/gateway.yaml
kubectl --context kind-country-assignment rollout status deployment/geocoding-gateway --timeout=180s
```

4. Deploy the REST application

```
kubectl --context kind-country-assignment apply -f k8s/deployment.yaml
kubectl --context kind-country-assignment rollout status deployment/country-lookup --timeout=180s
kubectl --context kind-country-assignment get deployment country-lookup
kubectl --context kind-country-assignment get pods -l app=country-lookup -o wide
kubectl --context kind-country-assignment get services
```

Expected after successful deployment: `country-lookup` shows two ready replicas. Show both application pods in the screencast. The extra gateway pod does not count as an application replica.

5. Access from a browser on the host

```
kubectl --context kind-country-assignment port-forward service/country-lookup 8081:8080
```

Leave this terminal open. In the Windows host browser, open:

```
http://localhost:8081/api/country?latitude=49.6116&longitude=6.1319
```

Also demonstrate invalid input:

```
http://localhost:8081/api/country?latitude=100&longitude=6.1319
```

To display HTTP status codes in another terminal:

```
curl.exe -i "http://localhost:8081/api/country?latitude=49.6116&longitude=6.1319"
curl.exe -i "http://localhost:8081/api/country?latitude=100&longitude=6.1319"
```

Port-forwarding exposes one selected service-backed pod for the demonstration; the deployment still runs two replicas. It does not demonstrate load distribution across both replicas.

Troubleshooting

```
kubectl --context kind-country-assignment describe deployment country-lookup
kubectl --context kind-country-assignment get events --sort-by=.metadata.creationTimestamp
kubectl --context kind-country-assignment logs deployment/country-lookup
kubectl --context kind-country-assignment logs deployment/geocoding-gateway
```

• `ErrImageNeverPull`: build/load the exact `country-lookup:1.0` image into this named cluster.

• Connection refused in browser: keep port-forward running and use host port 8081.

• 502 on valid coordinates: inspect connectivity, provider response and gateway logs. A public service may reject or fail requests.

• If image download or DNS resolution fails, check Docker Desktop and network access.

Cleanup

Stop port-forward with Ctrl+C. Delete only this assignment's local cluster when finished:

```
kind delete cluster --name country-assignment
```

Mandatory screencast

Unlisted YouTube video URL: Not recorded or uploaded yet.

Record yourself performing the Part 3 commands, showing:

1. Cluster creation and a ready node.

2. Docker image build and loading into kind.

3. Gateway and application deployment.

4. Two ready application pods.

5. A successful request from the host browser.

6. A 400 response for invalid coordinates.

Upload the recording to YouTube with visibility set to **Unlisted**, then replace the pending video statement above with the actual link. Parts 1 and 2 videos are optional. Do not submit this README as fully complete until the mandatory video and measured image size have been added.

Regenerate the PDF

After updating the README, regenerate the matching PDF:

```
python -m pip install reportlab
python scripts/readme_to_pdf.py
```

This overwrites `README.pdf` from `README.md`. Both formats must be included in the final submission.

Source layout

Path	Purpose
src/main/java/assignment1/demo/CountryController.java	Endpoint and coordinate validation
src/main/java/assignment1/demo/CountryService.java	WebClient lookup, response checks, timeout and cache
src/main/java/assignment1/demo/CountryResponse.java	Successful JSON response
src/main/java/assignment1/demo/ApiErrorHandler.java	JSON error responses
src/test/java/assignment1/demo/CountryApiTests.java	Tests with a local simulated provider
Dockerfile	Multi-stage application build
k8s/deployment.yaml	Two application replicas and Service
k8s/gateway.yaml	Shared provider gateway and Service
gateway/nginx.conf	Global provider rate limit and cache
scripts/readme_to_pdf.py	README PDF generation

References

• Spring Boot: https://docs.spring.io/spring-boot/

• Spring WebClient: https://docs.spring.io/spring-framework/reference/web/webflux-webclient.html

• Nominatim reverse API: https://nominatim.org/release-docs/latest/api/Reverse/

• Nominatim usage policy: https://operations.osmfoundation.org/policies/nominatim/

• OpenStreetMap attribution: https://www.openstreetmap.org/copyright

• Docker multi-stage builds: https://docs.docker.com/build/building/multi-stage/

• kind quick start: https://kind.sigs.k8s.io/docs/user/quick-start/

AI assistance was used to help prepare the implementation, tests and documentation. Review the code and follow the course's disclosure rules before submitting.